

---

# SYDE 522 Final Project

---

## Confused student EEG brainwave data

Yoonhee Park 20958776

Hassan Alawie 20780761

## Abstract

This report evaluates EEG brainwave data collected from college students watching STEM-related Massive Open Online Courses (MOOC) videos to classify whether a student was confused during a session. The dataset includes temporal EEG signals, attention and meditation scores, and demographic information, enabling us to test various machine learning and deep learning algorithms for binary classification. Initially, we preprocessed the data by normalizing features and experimented with an aggregated approach by averaging signals across video sessions to simplify the dataset for traditional models like SVM and KNN. Later, we explored sequential models, such as LSTMs and CNNs, leveraging their ability to capture temporal and spatial patterns in the raw EEG data. The results demonstrated the strengths of deep learning models, with the KNN and SVM achieving a test accuracy of 99%, the RNN achieving 100%, the CNN achieving 97% and the LSTM achieving 96%. Our findings highlight the importance of retaining temporal information in EEG data and optimizing model hyperparameters to balance generalization and computational efficiency.

## 1 Introduction

### 1.1 Description of Task and Data

This dataset contains EEG brainwave data collected from 10 college students watching STEM-related MOOC videos, categorized into confusing and non-confusing topics. The videos range from simpler topics, like basic limit calculations, to complex ones, such as biophysics and digital systems. Each student watched 20 videos (10 from each category), with EEG signals recorded over a one-minute duration measuring frontal lobe activity. The students self-rated their confusion levels on a scale of 1-7, which were further normalized into binary labels: "confused" or "not confused." The 'user-definedlabeln' column indicates whether a student was confused during a video, serving as the target for classification.

The EEG data includes columns representing various power spectrum bands (**Delta, Theta, Alpha, Beta, and Gamma**) and **attention and meditation** scores, measuring focus and calmness respectively. Each row corresponds to 0.5 seconds of recorded data. The **Raw** EEG signal was also provided for each user. Additionally, a demographic dataset provides student details like age, gender, and ethnicity and we would be able to join this dataset with the other using the subjects **SubjectId**.

The task is a binary classification problem, where the goal is to predict whether a student is confused (1) or not confused (0) during a video session based on their EEG signal data. Models would first pre-process the data, and then use features like the EEG power bands, proprietary scores, and potentially demographic information to predict confusion.

| SubjectID | VideoID | Attention | Mediation | Raw   | Delta     | Theta    | Alpha1   | Alpha2  | Beta1   | Beta2    | Gamma1  | Gamma2  | predefinedlabel | user-definedlabel |
|-----------|---------|-----------|-----------|-------|-----------|----------|----------|---------|---------|----------|---------|---------|-----------------|-------------------|
| 0.0       | 0.0     | 56.0      | 43.0      | 278.0 | 301963.0  | 90612.0  | 33735.0  | 23991.0 | 27946.0 | 45097.0  | 33228.0 | 8293.0  | 0.0             | 0.0               |
| 0.0       | 0.0     | 40.0      | 35.0      | -50.0 | 73787.0   | 28083.0  | 1439.0   | 2240.0  | 2746.0  | 3687.0   | 5293.0  | 2740.0  | 0.0             | 0.0               |
| 0.0       | 0.0     | 47.0      | 48.0      | 101.0 | 758353.0  | 383745.0 | 201999.0 | 62107.0 | 36293.0 | 130536.0 | 57243.0 | 25354.0 | 0.0             | 0.0               |
| 0.0       | 0.0     | 47.0      | 57.0      | -5.0  | 2012240.0 | 129350.0 | 61236.0  | 17084.0 | 11488.0 | 62462.0  | 49960.0 | 33932.0 | 0.0             | 0.0               |
| 0.0       | 0.0     | 44.0      | 53.0      | -8.0  | 1005145.0 | 354328.0 | 37102.0  | 88881.0 | 45307.0 | 99603.0  | 44790.0 | 29749.0 | 0.0             | 0.0               |

Figure 1: EEG Data Preview

## 1.2 Choice of Algorithms

When analyzing EEG data, we explored a range of machine learning and deep learning models to evaluate their performance on this challenging dataset. EEG signals are inherently noisy, time-dependent, and high-dimensional, making it crucial to test a diverse set of algorithms to determine which are best suited for such data [1]. We included traditional machine learning models like SVM, KNN, and Logistic Regression, which allowed us to explore the impact of varying hyperparameters like C, kernel function, and K, to observe how well they can capture the patterns in the data. These models are simpler and often perform well on small datasets, providing a solid baseline for comparison.

We also tested neural network models like LSTMs, RNNs, CNNs, and MLPs, which are particularly useful for capturing the temporal and spatial features of EEG signals [2]. LSTMs and RNNs, designed for sequential data, allowed us to analyze how the models' accuracy and loss evolved over training epochs, which is critical for time-series data like EEG. Similarly, CNNs provided insights into how spatial patterns in the EEG features could be extracted. For MLPs, we varied parameters like hidden layer sizes and activation functions to test their ability to model the complex relationships in the data. Logistic Regression, though simple, offered a benchmark for how well a linear model could separate the binary labels.

Testing these diverse models was worthwhile because EEG data presents unique challenges that different algorithms address in varied ways. This approach allowed us to confirm our hypothesis that not all models perform well on this type of data and helped identify which approaches hold the most promise for similar datasets in future applications.

## 2 Method 1 Aggregate Mean By Student and Video

One approach we took to handle the EEG data was to aggregate the time-based signals by calculating the mean of all fields for a given subject watching a specific video. This method effectively summarized the temporal data into a single *snapshot* per video-subject pair, transforming the dataset into a more simple format. By doing so, we eliminated the time dependency in the data, simplifying its structure and possibly making it more suitable for traditional machine learning models like Logistic Regression, SVM, and KNN, which are not inherently designed to capture temporal dynamics. We've learned of other models in class that are far better than these models at capturing time-based affects, but we still wanted to evaluate these other models.

Aggregating the features ensured that each instance in the dataset represented a overview of the subject's experience during a video, without the added computational burden of modeling time-series relationships. While this method might overlook finer temporal variations in the data, it provided a straightforward and efficient way to assess model performance and allowed us to compare results across different algorithms under much simplified conditions.

### 2.1 Pre-processing

To prepare the EEG dataset for analysis, we began by normalizing the signal features (Delta, Theta, Alpha1, etc.) and the Raw signal by dividing each value by the maximum value in its respective column. This step helped to address noticeable inconsistencies in the raw data, such as Subject 2's lower EEG values and Subject 6's higher "Raw" signal values. By normalizing, we ensured that the data was more evenly distributed across subjects, as demonstrated in Figure 2. Next, the normalized data was aggregated at the SubjectID and VideoID levels by calculating the mean for all numerical fields, removing temporal dependencies while preserving key information about the

subjects' responses to each video. The confusion label (user-definedlabeln) was retained as is to serve as the target variable for classification.

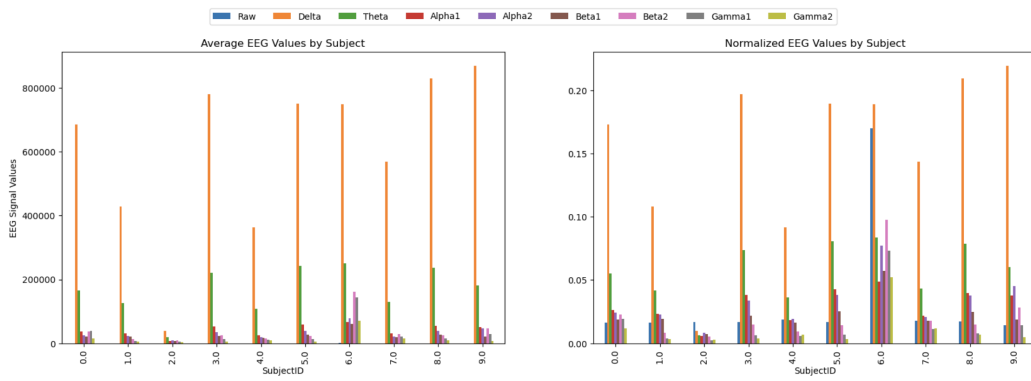


Figure 2: Effects of Normalization on Subject 2

We then merged the aggregated dataset with demographic information, such as age, gender, and ethnicity, to incorporate subject-specific characteristics into the analysis. To prepare the data for machine learning models, categorical fields like ethnicity and gender were one-hot encoded, while numerical features were standardized to ensure equal weighting across all variables. Finally, the processed dataset was split into training, test, and validation sets, creating a robust foundation for evaluating the performance of various predictive models. We limited our pre-processing there as we did not want to lose any more data, since we would have far less data points to train our models using this method.

## 2.2 Results

### 2.2.1 SVM

The initial SVM model with default parameters achieved an accuracy of 65%, but after hyperparameter tuning with grid search, the optimized model achieved a cross-validation accuracy of 82.5% using an RBF kernel. This demonstrates the importance of selecting appropriate hyperparameters, such as the kernel, C (regularization), and gamma (kernel coefficient). The kernel determines how the model maps the data in a higher-dimensional space, with linear kernels being simpler and computationally efficient, while RBF kernels handle complex relationships, as discussed in class. The regulation parameter (C) controls the tradeoff between margin size and tolerance for misclassification, where larger values risk overfitting, and smaller values may underfit. Gamma affects the complexity of the decision boundary, with higher values risking overfitting and lower values potentially oversimplifying the model.

To finalize the model's accuracy, the **training** set is used to fit the model and tune hyperparameters through cross-validation on the **validation** set. Once the best parameters are identified, the model is retrained on the entire training set (including the validation set) and evaluated on the **testing** set. This ensures the reported accuracy reflects the model's true performance on unseen data and avoids making it so that we avoid any bias introduced during hyperparameter tuning, in other words, tuning our parameters to make our model excel using the testing data.

### 2.2.2 KNN

In the initial attempt using KNN with K=5 and the default distance metric (Euclidean), the model achieved an accuracy of 65%, which served as a reasonable baseline. To further evaluate performance, we experimented with varying K values and different distance metrics, including Euclidean, Manhattan, Chebyshev, and Minkowski, as shown in Figure 3. This analysis highlighted how different K values and metrics impacted the accuracy, with certain combinations outperforming others. For instance, increasing K generally smoothed decision boundaries, reducing sensitivity to noise but risking underfitting if K was too high. Similarly, varying the distance metric revealed how different measures of similarity influence classification performance [3].

Following hyperparameter optimization, the best-performing combination of  $K$  and distance metric was tested on the unseen testing set, resulting in an improved accuracy of 75%. This highlights the importance of tuning parameters to balance overfitting and underfitting and ensuring the chosen settings generalize well to unseen data. We can even see in Figure 3, how the accuracy *generally* improves as we increase  $K$ , and it eventually levels out. Note that the Euclidean metric overlaps exactly with the Minkowski metric because, by default, the Minkowski metric is set with  $P = 2$ , which makes it the same as the Euclidean metric. The final accuracy demonstrates the robustness of KNN when appropriately configured, even for a complex dataset like EEG signals. It also demonstrates the positive affects of aggregating this time based data by 'SubjectID' and 'VideoID'.

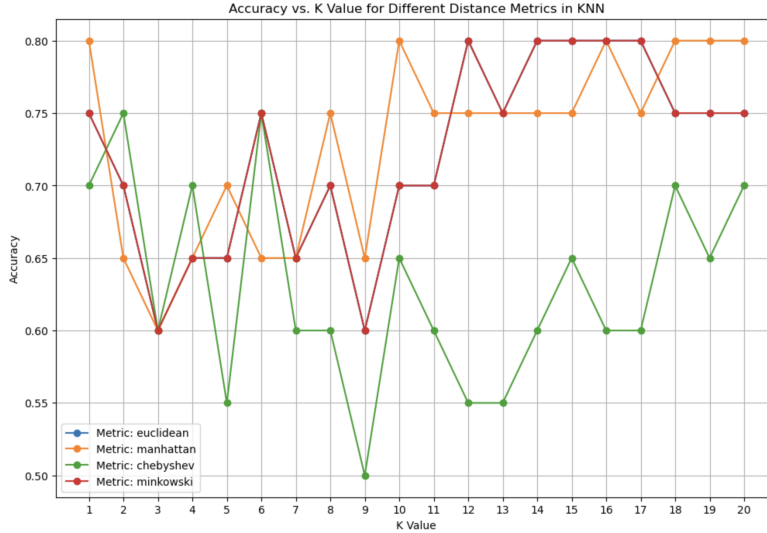


Figure 3: KNN Analysis varying  $K$  and Distance Measures

### 3 Method 2 Not Averaging Data

#### 3.1 Pre-processing

Similar to the method of averaging data, normalization was performed by excluding a subject whose data was significantly different from others and addressing outliers by capping them between the 1st and 99th percentiles. Given the large number of categories, we examined the correlation scores of these categories by plotting a Correlation Heatmap to identify the most relevant features. The analysis revealed that Delta, Theta, Alpha 1 & 2, Beta 1 & 2, and Gamma 1 & 2 are the most critical features to retain. Additionally, since features with high mutual information with the target variable can enhance the predictive capability of the algorithm [4], we assessed mutual information scores. Features such as 'age,' 'predefinedlabel,' and 'ethnicity' were removed from the X data due to their mutual information score of 0. This further justified using 'user-definedlabeln' as Y data instead of 'predefinedlabel' as Y data. To prevent the model from learning patterns based on IDs, both 'SubjectID' and 'VideoID' were removed. Similarly, 'gender' data was excluded to avoid introducing bias, as this project does not aim to categorize based on gender.

#### 3.2 Results

##### 3.2.1 SVM

Using `sklearn.svm.SVC` with a second-order polynomial kernel, a regularization parameter  $C$  set to 2, and all other settings at their defaults, the test accuracy achieved was 83%. To improve model performance, we experimented with higher regularization parameters. Increasing  $C$  to 5 and 10 resulted in test accuracies of 93.5% and 100%, respectively. However, the downside of using second-order polynomials is the high computational cost. To address this, we tested the RBF kernel to reduce

run-time. With significantly faster execution, the RBF kernel achieved a test accuracy of 99.9% when C was set to 2. However, increasing C further did not yield any additional improvement in accuracy.

### 3.2.2 KNN

Continuing the investigation from the previous section for KNN, it is evident that selecting the Manhattan metric with a K value of at least 8 is a reliable choice, as shown in Figure 4.

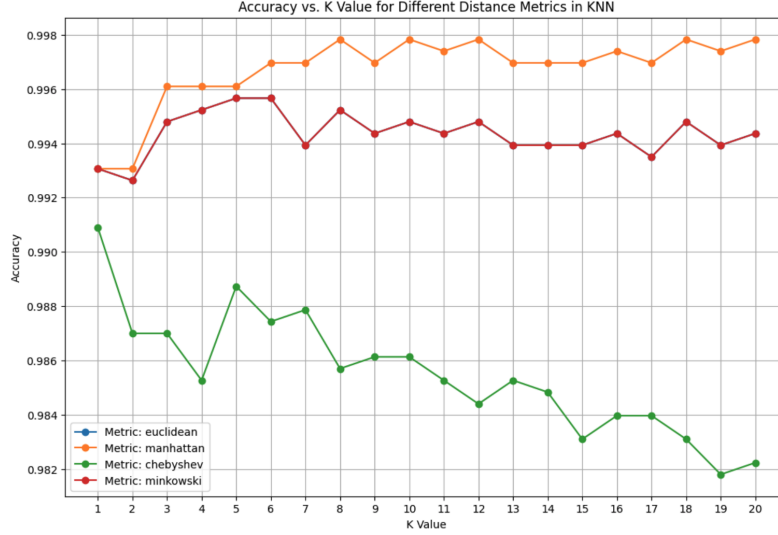


Figure 4: KNN Analysis varying K and Distance Measures

## 4 Further Investigating the Non-Averaging Method

Using the complete sequential data is often better for analyzing EEG signals because it preserves the temporal dynamics and patterns critical to understanding the brain's activity over time. EEG data is inherently time-dependent, and is often used to capture changes in frequencies that occur at specific moments [1], like perhaps a student watching a video and becoming more confused as it goes on. Perhaps a student was confused for the majority of the video, but by the end, they finally understood what was happening. By averaging the data using the mean, these temporal variations are lost, resulting in a singular value or group of values that may obscure some really valuable insights. For instance, in aggregating data, we lose information regarding the rate at which spikes may occur, which can be very valuable in our interpretation of EEG signals [5]. Additionally, temporal relationships across features, such as how one frequency band might influence another over time, are completely disregarded. Models designed for sequential data, such as LSTMs or RNNs, leverage the valuable insights we gain from temporal data to learn patterns and dependencies that static approaches cannot capture [2], ultimately resulting in more nuanced and accurate predictions.

### 4.1 RNN

Starting with an 'adam' optimizer and a sequential model containing a SimpleRNN layer with 128 units representing the hidden states in RNN cells, the accuracy and loss of the model vary with each run, as shown in Figure 5 and Figure 6.

This behavior is expected because the 'sparse\_categorical\_crossentropy' loss function is designed for multiclass classification, whereas the task requires a binary classification approach. To address this, the model was modified for binary classification by adding a dense layer with a single output and sigmoid activation, along with the 'BinaryCrossentropy' loss function. This adjustment achieved 100% accuracy early in the training epochs, as illustrated in Figure 7, resulting in both high accuracy and low accuracy in each trial.

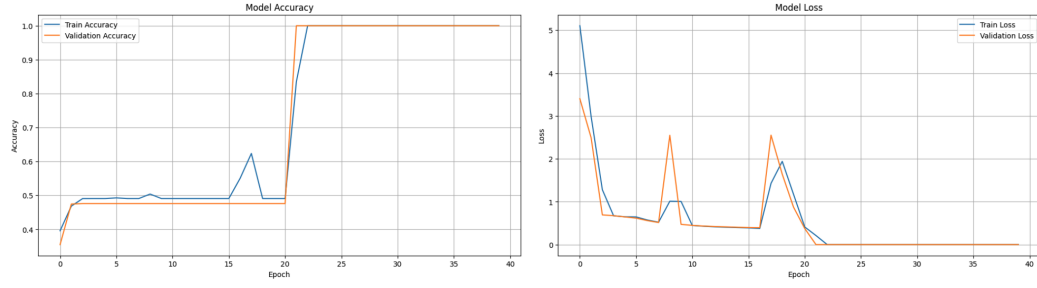


Figure 5: RNN Multiclass Accuracy and Loss Trial 1

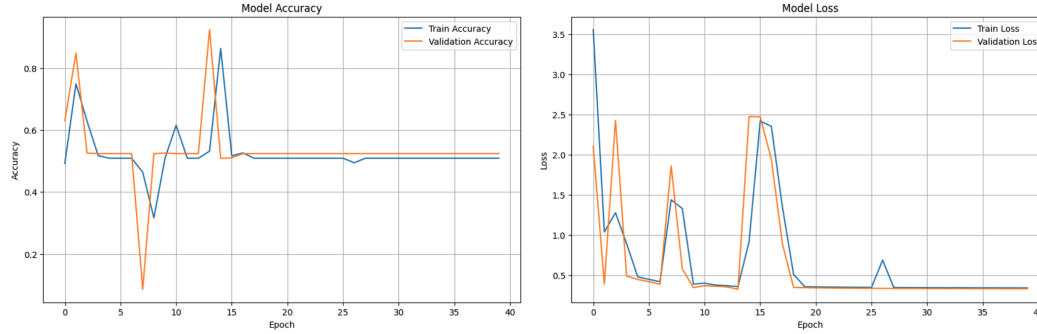


Figure 6: RNN Multiclass Accuracy and Loss Trial 2

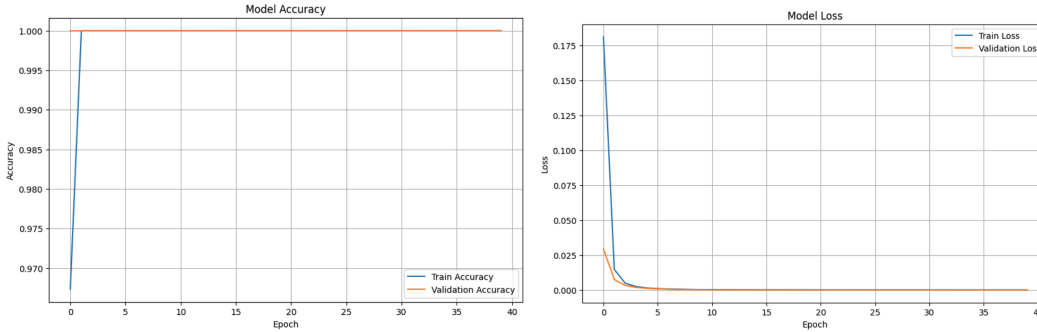


Figure 7: RNN Binary Classification Accuracy and Loss

## 4.2 LSTM

Using an LSTM model, we leveraged the complete sequential EEG data without aggregating the entire dataset, allowing the network to capture temporal dependencies critical for EEG data. The model achieved a final test accuracy of 96%, significantly higher than previous approaches using aggregated data. This result highlights the strength of LSTMs in preserving and learning from time-series data. Training and validation curves (Figure 8) showed consistent improvement in accuracy over 20 epochs, with the validation accuracy reaching 98% and validation loss stabilizing, which was an indicator to us that increasing the number of epochs from there would have less of an effect.

When varying batch size, we noticed that increasing the batch size from 32 to 64 led to much faster convergence, but the model performed more poorly, potentially indicating that increasing the batch size smoothed out critical updates over time. This highlights how adjusting batch size provides a balance between computational efficiency and model performance, reinforcing the importance of careful hyperparameter tuning for optimal results.

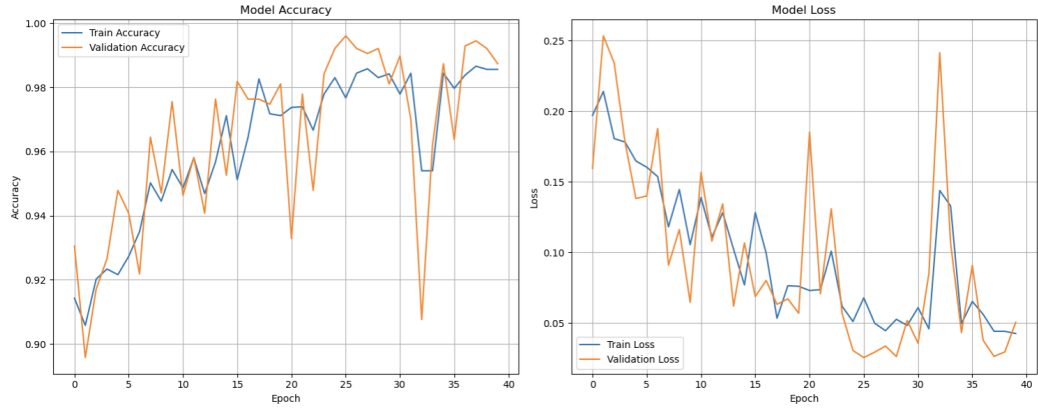


Figure 8: LSTM Model Accuracy and Loss

### 4.3 CNN

The CNN, like the LSTM, was designed to analyze the sequential EEG data but focused on extracting spatial patterns across features using convolutional layers. This approach allowed the model to identify localized relationships in the data, complementing the temporal focus of the LSTM.

The CNN achieved a test accuracy of 97%, only slightly surpassing the LSTM. As shown in Figure 6, the training and validation accuracy converged near 99% with low loss values over 40 epochs. These results demonstrate the CNN's ability to effectively capture spatial patterns in the EEG data while maintaining strong generalization.

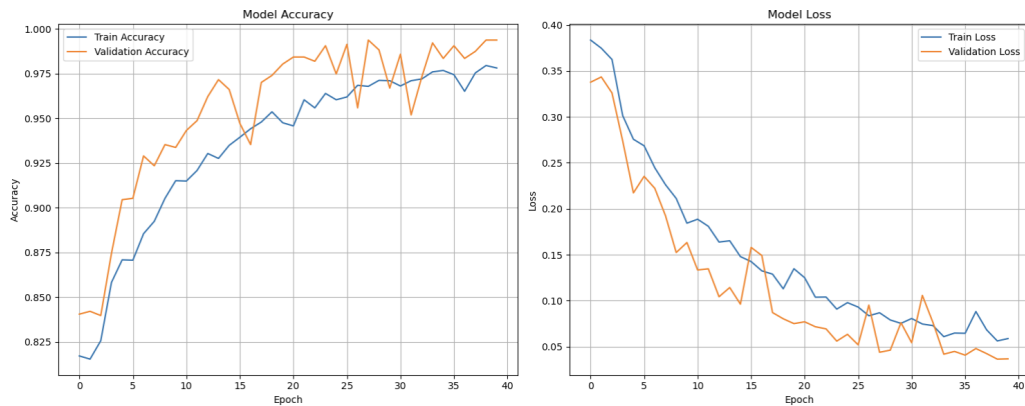


Figure 9: CNN Model Accuracy and Loss

## References

- [1] Gourav Siddhad, P. P. Roy, and B.-G. Kim, "Neural Networks Meet Neural Activity: Utilizing EEG for Mental Workload Estimation," Lecture notes in computer science, pp. 325–339, Dec. 2024, doi: [https://doi.org/10.1007/978-3-031-78195-7\\_22](https://doi.org/10.1007/978-3-031-78195-7_22).
- [2] Branislav Radomirovic et al., "Optimizing long-short term memory neural networks for electroencephalogram anomaly detection using variable neighborhood search with dynamic strategy change," Complex & Intelligent Systems, Aug. 2024, doi: <https://doi.org/10.1007/s40747-024-01592-z>.
- [3] H. Arafat et al., "Effects of Distance Measure Choice on K-Nearest Neighbor Classifier Performance: A Review," Big Data, vol. 7, no. 4, pp. 221–248, Aug. 2019, doi: <https://doi.org/10.1089/big.2018.0175>.

- [4] Huang, L., Zhou, X., Shi, L., & Gong, L. (2024). Time Series Feature Selection Method Based on Mutual Information. *Applied Sciences*, 14(5), 1960. <https://doi.org/10.3390/app14051960>
- [5] S. K. R. Singanamalla and C.-T. Lin, "Spike-Representation of EEG Signals for Performance Enhancement of Brain-Computer Interfaces," *Frontiers in Neuroscience*, vol. 16, Apr. 2022, doi: <https://doi.org/10.3389/fnins.2022.792318>.